Axolotl 0.3.0 on macOS

Installing Axolotl and working with the axl command line compiler

1. What you get

Axolotl is a JVM language that translates to readable Java 21. Its compiler checks that values which may be null are handled, using NullAway, and moves every error back to the line of the `.axl` file.

The installer does three things:

- installs the `axl` command together with its own Java 21 runtime, so `axl` needs no Java on the computer. The runtime has every module of Java SE, AWT and Swing included;
- puts `axl` on the `PATH`;
- runs `axl setup`, which copies the Axolotl libraries and the Gradle and Maven plugins into the local Maven repository, so that Gradle and Maven projects find them.

2. Requirements

- A Mac with Apple silicon (M1 or newer) and a current macOS.
- On an Intel Mac use `axl-0.3.0.zip` from the release page instead, with a JDK 21 installed: unzip it, put its `bin` directory on the `PATH` and run `axl setup`.
- An administrator password for the installation.
- Gradle and Maven, if you use them, run on a JDK 21 of their own: the runtime inside `axl` is only for `axl`.

3. Installation

1. Download `axolotl-0.3.0-macos-arm64.pkg` from https://github.com/succubus-uwu/axolotl/releases/tag/v0.3.0.
2. Open it. The package is not signed, so macOS may refuse to open it. Then open *System Settings* > *Privacy & Security*, scroll down and click *Open Anyway*, and open the package again.
3. Follow the installer and enter your password when it asks.

Instead of step 2 you can remove the quarantine mark in Terminal first:

```
$ xattr -d com.apple.quarantine ~/Downloads/axolotl-0.3.0-macos-arm64.pkg
```

The package installs `/Applications/axl.app`, links `/usr/local/bin/axl` to it and runs `axl setup` for the user who is logged in. `/usr/local/bin` is on the `PATH` of Terminal by default. From the command line the same installation is:

```
$ sudo installer -pkg ~/Downloads/axolotl-0.3.0-macos-arm64.pkg -target /
```

4. Checking the installation

```
$ axl --version
axl 0.3.0
```

`axl --help` lists the commands, and `axl help <command>` or `axl <command> --help` describes one.

5. The first program

Sources live in a source directory, with one directory per package part, as in Java. Make a project directory with `src/app/Main.axl` in it:

```
package app

import java.util.List

fn greet(name: String?) -> String = "Hello, " + (name?.trim() ?: "stranger")

fn main() {
    val words = List.of(args)
    println(greet(if (words.isEmpty()) null else words.get(0)))
}
```

Run it from the project directory. Everything after the file name goes to the program as `args`:

```
$ axl run src/app/Main.axl
Hello, stranger
$ axl run src/app/Main.axl Ann
Hello, Ann
```

`axl run` finds the source directory from the package of the file, translates and compiles every `.axl` and `.java` file in it, checks nulls and runs `fn main()`. Save the files in UTF-8.

6. The commands

Command	What it does
`axl run <file> [args...]`	Compiles the source tree of the file and runs its `fn main()`.
`axl check <dirs...>`	Translates and compiles, keeps no output and prints the errors. Fast feedback, and what editors call.
`axl build -d <dir> <src>`	Translates and compiles a source directory into class files.
`axl retrace [file]`	Rewrites the `.java` lines of an old stack trace to `.axl` lines.
`axl setup`	Installs the libraries and plugins into the local Maven repository.
`axl help [command]`	Describes the commands.

Options

Option	Commands	Meaning
`-cp`, `--classpath`	run, check, build, retrace	Jars and class directories the code uses, separated by `:`.
`-d <dir>`	build	Where the class files go.
`-s <dir>`	build	Where the generated `.java` and `.axlmap` files go (default: the `-d` directory).
`--strict-null`	run, check, build	Treats values from Java code without nullness annotations as nullable.
`--no-null-checks`	run, check, build	Skips Error Prone and NullAway: faster, but nulls are not checked.
`--inline-elvis`	run, check, build	Expands `?:` at statement level without a lambda.
`--no-param-checks`	run, check, build	Leaves out the null checks of public functions and constructors against callers from Java.
`--no-smap`	build	Keeps the line numbers of the generated `.java` files in the classes.
`-m`, `--map-dir`	retrace	A directory with generated `.java` and `.axlmap` files; repeatable.
`--repository`	setup	The local Maven repository to install into.
`-h`, `-V`	all	Help and version.

7. Checking code: axl check

Add a file that forgets a null check, `src/app/Broken.axl`:

```
package app

fn shout(name: String?) -> String = name.toUpperCase()
```

```
$ axl check src
src/app/Broken.axl:3:42: error: value `name` may be null: use `?.`, `?:` or check `if (name
    != null)` [AXL102]
```

Every message has the form `file:line:column: error|warning: text [code]`. The codes are listed in `docs/errors.md`: AXL0xx come from Axolotl's own checks, AXL1xx from javac, Error Prone and NullAway, reworded in Axolotl terms. Warnings do not stop a build.

`axl check`, `axl build` and `axl run` exit with code 1 when there are errors and 0 otherwise, so scripts and CI can test `$?`. `axl check` takes several source directories, such as `axl check src/main src/test`.

Fix it with `name?.toUpperCase() ?: ""` and `axl check src` prints nothing.

8. Building class files: axl build

```
$ axl build -d out/classes -s out/gen src
```

`out/classes` gets the class files, `out/gen` the generated Java and the `.axlmap` line maps. The generated Java is plain Java 21 you can read. Code that uses other libraries passes them with `-cp`, separated by `:`.

The classes run on any Java 21 or newer with the Axolotl runtime library on the classpath. `axl setup` put it into the local Maven repository. `axl` does not provide a `java` command, so this needs a Java of your own:

```
$ RT=~/.m2/repository/org/axl/axl-runtime/0.3.0/axl-runtime-0.3.0.jar
$ java -cp "out/classes:$RT" app.Main Bob
Hello, Bob
```

For anything larger than a few files a Gradle or Maven build is simpler (section 10).

9. Stack traces

Classes built by `axl build` name the `.axl` file and line themselves, so a program started with `java` prints them in every stack trace and log:

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index: 5 Size: 1
    ...
    at app.Main.first(Main.axl:5)
    at app.Main.main(Main.axl:8)
```

`axl retrace` is for traces that still name the generated Java, such as old logs or classes built with `--no-smap`. It reads a trace from a file or from standard input and points it at the `.axl` lines using the `.axlmap` files (`-m out/gen`):

```
    at app.Main.first(Main.java:9)
    at app.Main.main(Main.java:12)
```

```
$ axl retrace -m out/gen trace.txt
    at app.Main.first(Main.axl:5)
    at app.Main.main(Main.axl:8)
```

10. Gradle and Maven projects

After `axl setup` both build tools find Axolotl in the local Maven repository. They still download Error Prone, NullAway and the other dependencies from Maven Central, and need a JDK 21.

Gradle

```
// settings.gradle.kts
pluginManagement { repositories { mavenLocal(); gradlePluginPortal() } }
dependencyResolutionManagement { repositories { mavenLocal(); mavenCentral() } }

// build.gradle.kts
plugins {
    application
    id("org.axl.lang") version "0.3.0"
}
application { mainClass = "app.Main" }
```

Sources go to `src/main/axl` and `src/test/axl`; `gradle run` builds and runs the program.

Maven

A new project from the archetype, ready to build with `mvn package`:

```
$ mvn archetype:generate \
    -DarchetypeGroupId=org.axl -DarchetypeArtifactId=axl-archetype-quickstart \
    -DarchetypeVersion=0.3.0 \
    -DgroupId=org.example -DartifactId=demo -Dpackage=org.example.demo
```

11. Updating and uninstalling

Open the package of the new version; it replaces `/Applications/axl.app` and runs `axl setup`.

- In Terminal:

```
$ sudo rm -rf /Applications/axl.app /usr/local/bin/axl
$ sudo pkgutil --forget org.axl.cli
```

The libraries `axl setup` installed stay in the local Maven repository under `org/axl`; delete that directory too if no project needs them.

12. Troubleshooting

Problem	What to do
zsh: command not found: axl.	Check that `/usr/local/bin/axl` exists and that `/usr/local/bin` is on the `PATH`: add `export PATH="/usr/local/bin:$PATH"` to `~/.zshrc`.
macOS refuses to open the package.	*System Settings* > *Privacy & Security* > *Open Anyway*, or `xattr -d com.apple.quarantine` on the package (section 3).
Another user of the Mac needs the Maven libraries.	That user runs `axl setup` once.
Maven or Gradle cannot find `org.axl.lang` or `axl-maven-plugin`.	Run `axl setup` as the user who builds; check that the build lists `mavenLocal()`. If you set `-Dmaven.repo.local` in `MAVEN_OPTS`, run `axl setup` again from that shell, or pass `axl setup --repository <dir>`.
`java` is not found, or is older than 21, when running built classes or Gradle.	Install a JDK 21; `axl` keeps its own runtime to itself.
A compile takes long the first time.	`axl` starts a JVM and loads javac and NullAway on every call; a few seconds are normal. `--no-null-checks` skips NullAway for quick experiments.

More

The documentation of the language, the Gradle and Maven plugins and the editors is at https://github.com/succubus-uwu/axolotl/tree/v0.3.0/docs. Releases and installers: https://github.com/succubus-uwu/axolotl/releases.